

Phase 2 — Technical Documentation

ESL Orchestrator

10.04.2026

João Pires - Devoteam

Sonae MC

Contents

1. Introduction	1
1.1. Purpose	1
1.2. Scope	1
1.3. Audience	1
1.4. How to Read This Document	1
1.5. Glossary	2
2. System Architecture	2
2.1. Overview	2
2.2. Architecture	4
2.3. CDC Data Flow	6
2.4. Component Responsibilities	6
2.5. Shared Library: esl-common	6
2.5.1. Packages	7
2.6. Key Design Decisions	7
2.6.1. Transactional outbox over CDC triggers	7
2.6.2. Pre-fetch and in-Go classification	8
2.6.3. Separate publisher service	8
2.6.4. Horizontal scaling of the Event Publisher	8
2.6.5. Feature flag	8
3. Database	8
3.1. Overview	8
3.2. Event Outbox Table	9
3.2.1. event_outbox	9
3.2.2. Payload format	10
3.2.3. Comparison scope	11
3.3. Indexes	11
3.4. Retention	12
3.5. Migrations	12
3.6. Key Design Decisions	13
3.6.1. Single outbox table for all entity types	13
3.6.2. JSONB for entity_key and payload	13
3.6.3. UUID primary key over sequential integer	13
3.6.4. Explicit status column over NULL-based tracking	13
3.6.5. Partial indexes over full indexes	13
3.6.6. Retention via function, not trigger	13
4. Data Pipeline — CDC	14
4.1. Overview	14

4.2.	Feature Flag	14
4.3.	Architecture	14
4.3.1.	CDC Struct	15
4.3.2.	Shared Helpers	15
4.4.	Change Detection	16
4.4.1.	Step 1 — Group by Entity Type	16
4.4.2.	Step 2 — Fetch Existing State	16
4.4.3.	Step 3 — Classify and Diff	16
4.4.3.1.	Column handling	17
4.4.3.2.	Value normalisation	17
4.4.3.3.	Timestamp handling in payloads	18
4.5.	Outbox Write	18
4.6.	Transaction Flow	18
4.6.1.	Single-writer assumption	19
4.7.	Performance	19
4.7.1.	Network round-trips	19
4.7.2.	Measured overhead	19
4.8.	Error Handling	20
4.8.1.	Fail-fast semantics	20
4.8.2.	Error classification	20
4.9.	Observability	20
4.9.1.	Metrics	20
4.9.2.	Logging	20
4.10.	Configuration Reference	21
4.11.	Known Limitations	21
4.11.1.	No DELETED event detection	21
5.	Event Publisher	22
5.1.	Overview	22
5.2.	Technology Stack	24
5.3.	Architecture	24
5.4.	Polling Loop	25
5.4.1.	1. Check for pending events	25
5.4.2.	2. Fetch and lock rows	25
5.4.3.	3. Process each row	25
5.4.4.	4. Mark outcomes	25
5.4.5.	5. Commit	26
5.5.	Event Transformation	26
5.5.1.	Payload assembly	26
5.5.2.	Store ID splitting	26
5.5.3.	Example published event	26
5.6.	Solace Publishing	27

5.6.1. Topic structure	27
5.6.2. Delivery mode	27
5.7. Health Endpoints	27
5.8. Graceful Shutdown	28
5.9. Configuration Reference	28
5.9.1. Database	28
5.9.2. Solace	29
5.9.3. Publisher	29
5.9.4. Health	29
5.9.5. Logging	29
5.10. Observability	30
5.10.1. Application metrics	30
5.10.2. SDK metrics	30
5.10.3. Logging	30
5.11. Delivery Guarantees	30
5.12. Key Design Decisions	31
5.12.1. Polling over WAL logical replication	31
5.12.2. Transaction-scoped publishing	31
5.12.3. No application-level retry for failed events	31
5.13. Known Limitations	32
5.13.1. No per-entity ordering guarantee	32
5.13.2. Failed events require manual intervention	32

1. Introduction

1.1. Purpose

This document provides the technical documentation for Phase 2 of the ESL Orchestrator platform at Sonae MC. Building on the data synchronisation pipeline established in Phase 1, Phase 2 adds Change Data Capture (CDC) — the ability to detect when entity records are created or modified during each pipeline execution and publish those changes as events to a Solace message broker.

Phase 2 delivers three new components and modifies one existing component:

Component	Role
esl-common (new)	Shared Go library containing CDC event types and entity key definitions, imported by both the Data Pipeline and the Event Publisher
Database — event outbox (new migrations)	Transactional outbox table that stores detected change events atomically alongside data upserts
Data Pipeline — CDC (modified)	Pre-fetch and in-Go classification logic added to the existing sink, writing change events to the outbox within the same database transaction as upserts
Event Publisher (new)	Long-running service that polls the outbox table and publishes events to Solace with guaranteed delivery

1.2. Scope

This document covers the architecture, design, and database changes introduced by Phase 2. Sections on the Data Pipeline CDC logic, Event Publisher service, deployment, and operations will be added incrementally as those components are implemented.

DELETED event detection is deferred to a future iteration.

1.3. Audience

This document is intended for the same audience as Phase 1:

- Technical stakeholders who need to understand the CDC architecture and its integration with the existing platform
- Developers who will extend or maintain the CDC pipeline and Event Publisher
- Operations teams who will deploy, configure, and monitor the new components
- Project managers and product owners who need visibility into the platform's new capabilities

1.4. How to Read This Document

The document is structured in the following order:

1. System Architecture — Updated high-level view showing how CDC fits into the existing platform
2. Database — Event outbox table, indexes, and retention
3. Data Pipeline — CDC detection logic in the sink
4. Event Publisher — Outbox polling and Solace publishing
5. Deployment & Infrastructure — Helm chart changes for the new components (*to be added*)
6. Operations & Maintenance — Monitoring, outbox management, and troubleshooting (*to be added*)

1.5. Glossary

Terms introduced in Phase 1 still apply. The following terms are new in Phase 2:

Term	Description
CDC	Change Data Capture — detecting and recording data changes as they occur
Transactional Outbox	Pattern where change events are written to a database table within the same transaction as the data change, guaranteeing atomicity
Event Publisher	Service that polls the outbox table and publishes events to a message broker
Solace	Message broker used for event distribution to downstream consumers
Outbox	The event_outbox table that acts as a reliable buffer between change detection and event publishing
CREATED event	Event emitted when a record is inserted for the first time; payload contains the full record snapshot
UPDATED event	Event emitted when an existing record's non-audit fields change; payload contains only the changed fields as {"field": {"old": X, "new": Y}} diffs
Entity key	Composite business key that uniquely identifies a record (e.g., retail_chain_id + store_id for stores)
esl-common	Shared Go library providing CDC types and entity definitions used by multiple components

2. System Architecture

2.1. Overview

Phase 2 extends the ESL Orchestrator with a CDC (Change Data Capture) pipeline that detects data changes during each sync and publishes them as events. The existing Phase 1 architecture — ingest, store, serve — remains unchanged. Phase 2 adds a fourth stage: detect and publish.

The CDC pipeline uses the transactional outbox pattern: change events are written to an outbox table within the same database transaction as the data upserts, eliminating dual-write consistency problems. A separate Event Publisher service polls the outbox and delivers events to Solace.

2.2. Architecture

ESL Orchestrator — Phase 2 Architecture

Phase I (unchanged)

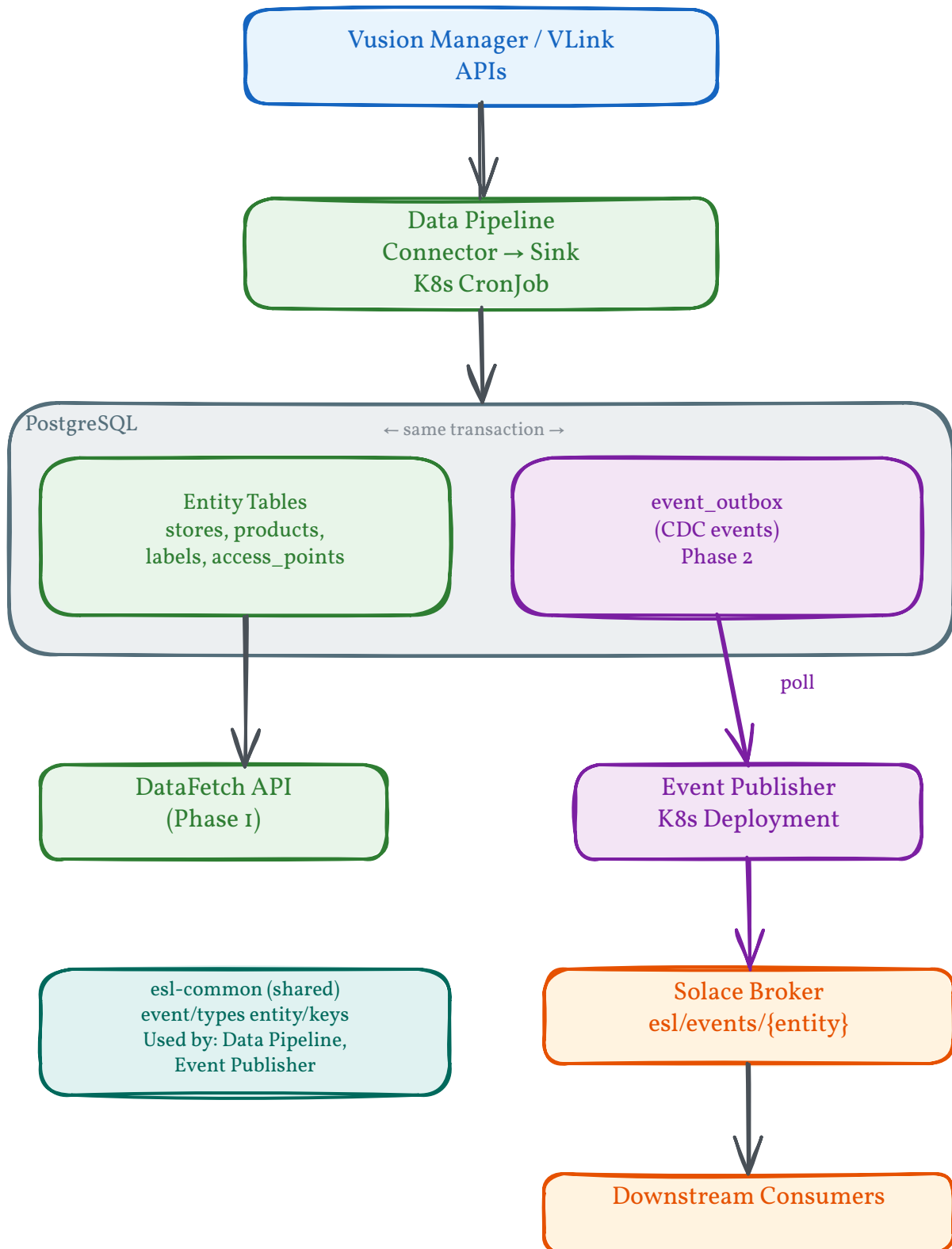


Figure 1: Phase 2 Architecture

2.3. CDC Data Flow

The CDC data flow adds three steps to the existing upsert path:

1. Pre-fetch — Before upserting a batch, the sink queries the current state of the target rows within a database transaction
2. Classify — Each incoming record is compared against the pre-fetched state:
 - Key not found → CREATED (payload: full record snapshot)
 - Key found, fields differ → UPDATED (payload: changed fields as old/new diffs)
 - Key found, identical → no event
3. Outbox write — Detected change events are inserted into `esl.event_outbox` within the same transaction as the upsert batch
4. Publish — The Event Publisher polls undelivered outbox rows, publishes each to Solace, and marks them as delivered

The transactional outbox guarantees that change events are only recorded when the corresponding data changes are committed. If the transaction rolls back, both the upsert and the outbox writes are discarded.

2.4. Component Responsibilities

Component	Type	Technology	Responsibility
Data Pipeline	CronJob	Go, pgx	Fetch data from Vusion APIs, transform, upsert, and detect changes (CDC)
Database	Job (PreSync)	PostgreSQL 18, Flyway 12	Schema management, data storage, event outbox
DataFetch API	Deployment	Go, Gin, pgx	Read-only REST API (unchanged from Phase 1)
Event Publisher	Deployment	Go, pgx, Solace SDK	Poll outbox, publish events to Solace, mark delivered
esl-common	Go library	Go	Shared CDC event types and entity key definitions

2.5. Shared Library: esl-common

The `esl-common` module provides types and definitions shared between the Data Pipeline and Event Publisher, ensuring consistent serialisation and entity key handling.

Module: github.com/sonaemc-instore/lac1041-instoreorchestrator_esl-common

2.5.1. Packages

event — CDC event types:

```
type ChangeType string
const (
    ChangeTypeCreated ChangeType = "CREATED"
    ChangeTypeUpdated ChangeType = "UPDATED"
)

type ChangeEvent struct {
    EventType ChangeType      `json:"event_type"`
    EntityType string            `json:"entity_type"`
    EntityKey  map[string]string          `json:"entity_key"`
    Payload    map[string]any             `json:"payload"`
    OccurredAt time.Time                 `json:"occurred_at"`
}
```

entity — Entity type enumeration and composite business keys:

```
type EntityType string
const (
    Store      EntityType = "store"
    Product    EntityType = "product"
    Label      EntityType = "label"
    AccessPoint EntityType = "accesspoint"
)

var ConflictKeys = map[EntityType][]string{
    Store:      {"retail_chain_id", "store_id"},
    Product:    {"retail_chain_id", "store_id", "item_id"},
    Label:      {"store_id", "retail_chain_id", "label_id"},
    AccessPoint: {"retail_chain_id", "store_id", "id"},
}
```

These conflict keys serve a dual purpose: they define the `ON CONFLICT` columns for upsert queries and the composite key included in each change event's `entity_key` field.

2.6. Key Design Decisions

2.6.1. Transactional outbox over CDC triggers

Database-level triggers (e.g., `LISTEN/NOTIFY` or logical replication) were considered but rejected in favour of application-level change detection with a transactional outbox. The outbox approach keeps all CDC logic in Go — where it can be tested, feature-flagged, and deployed independently — while guaranteeing atomicity with the upsert.

2.6.2. Pre-fetch and in-Go classification

Detecting changes requires knowing the previous state of each record. Since UPDATED events carry a diff payload (`{"field": {"old": X, "new": Y}}`), pre-fetching the current rows is mandatory. Given that pre-fetch is already needed, classification (CREATED vs UPDATED vs unchanged) is performed in Go at negligible additional cost.

2.6.3. Separate publisher service

The Event Publisher runs as its own Deployment rather than being embedded in the Data Pipeline CronJob. This decouples publish latency from sync execution, allows independent scaling and restarts, and means a Solace outage does not block data synchronisation.

2.6.4. Horizontal scaling of the Event Publisher

The Event Publisher uses `SELECT FOR UPDATE SKIP LOCKED` to claim outbox rows. This naturally supports multiple replicas — PostgreSQL skips rows already locked by other transactions, so each replica automatically gets a distinct set of rows with no coordination needed. The only caveat is event ordering per entity: with multiple replicas, two events for the same entity could be picked up by different replicas and published out of order. If downstream consumers require per-entity ordering, this would need partitioning by entity key (e.g. consistent hashing). For now, a single replica is sufficient.

2.6.5. Feature flag

CDC is gated behind a `cdc.enabled` configuration flag in the sink. When disabled, the pipeline behaves exactly as in Phase I — no transaction wrapper, no pre-fetch, no outbox writes.

3. Database

3.1. Overview

Phase 2 adds a single new table — `event_outbox` — to the existing `esl` schema. This table implements the transactional outbox pattern: the Data Pipeline inserts change events into the outbox within the same database transaction as entity upserts, and the Event Publisher polls the outbox to deliver events to Solace.

All existing Phase I tables remain unchanged.

esl schema — Entity Relationships (Phase 2)

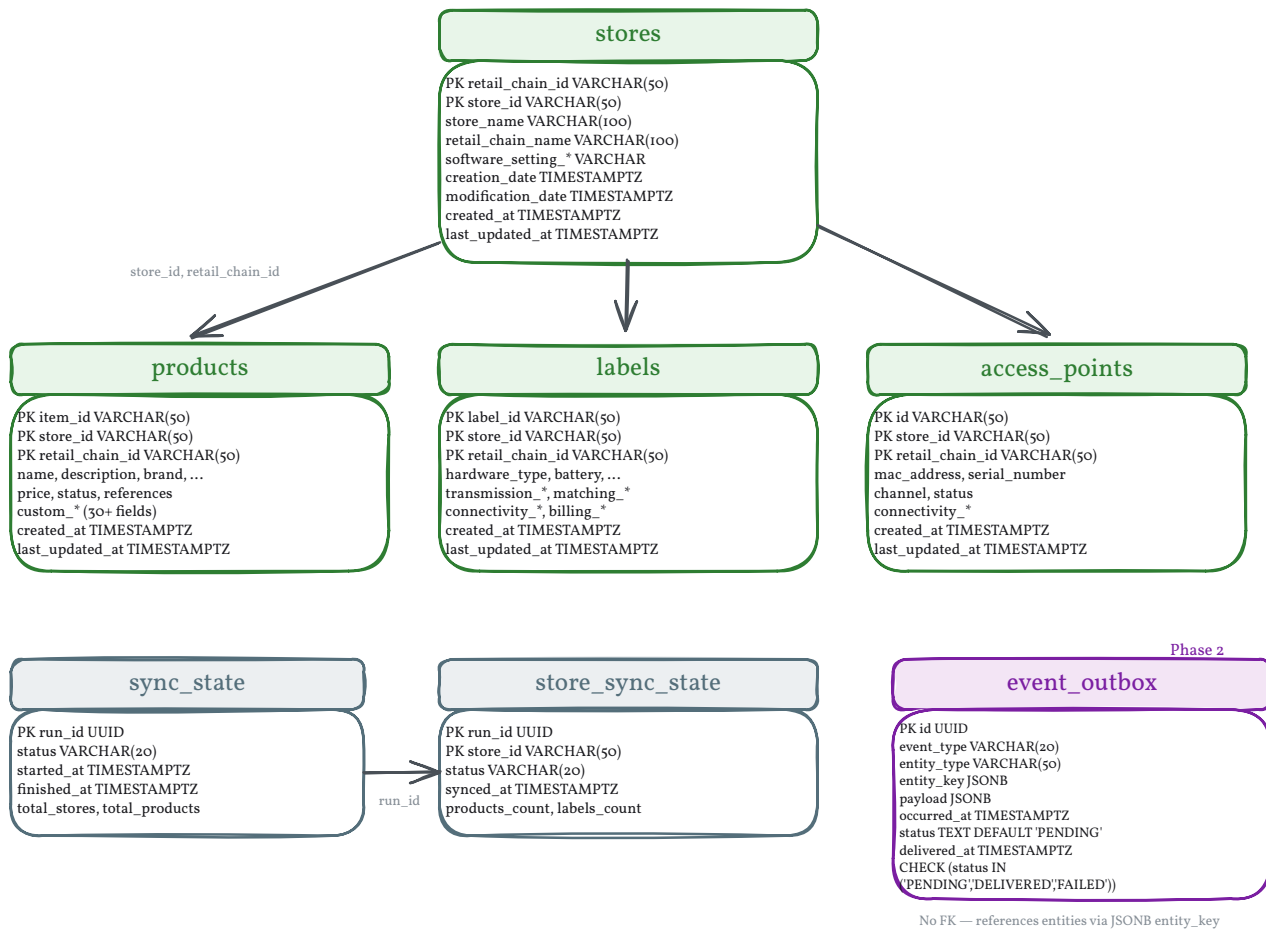


Figure 2: ER Diagram

3.2. Event Outbox Table

3.2.1. event_outbox

Stores CDC events detected during pipeline execution. Each row represents a single CREATED or UPDATED event for one entity record.

Column	Type	Default	Description
id	UUID	gen_random_uuid()	Unique event identifier (PK)
event_type	VARCHAR(20)	—	Change type: CREATED or UPDATED
entity_type	VARCHAR(50)	—	Entity that changed: store, product, label, accesspoint
entity_key	JSONB	—	Composite business key identifying the record (e.g., {"retail_chain_id": "RC001", "store_id": "S001"})
payload	JSONB	—	Event payload — full snapshot for CREATED, changed-field diffs for UPDATED
occurred_at	TIMESTAMPTZ	NOW()	When the change was detected (defaults to transaction time)
status	TEXT	'PENDING'	Delivery status: PENDING, DELIVERED, or FAILED
delivered_at	TIMESTAMPTZ	NULL	When the event was published to Solace

Primary key: id (UUID, database-generated)

Constraint: CHECK (status IN ('PENDING', 'DELIVERED', 'FAILED'))

The id column uses UUIDs rather than sequential integers. This enables deterministic event identifiers that can be included in published messages for downstream deduplication — consumers receiving the same event twice (at-least-once delivery) can use the UUID to discard duplicates.

The status column tracks delivery lifecycle explicitly. The Event Publisher transitions events from PENDING to DELIVERED (with delivered_at timestamp) after successful publication, or to FAILED for permanent errors. This explicit status model enables straightforward observability — a simple GROUP BY status reveals the outbox health at a glance.

3.2.2. Payload format

CREATED events carry a full snapshot of the record:

```
{
  "event_type": "CREATED",
  "entity_type": "product",
  "entity_key": {"retail_chain_id": "RC001", "store_id": "S001", "item_id": "P001"},
  "payload": {
    "name": "Leite Mimosa 1L",
    "price": 1.29,
    "status": "active",
    ...
  }
}
```

UPDATED events carry only the changed fields, with old and new values:

```
{
  "event_type": "UPDATED",
  "entity_type": "product",
  "entity_key": {"retail_chain_id": "RC001", "store_id": "S001", "item_id": "P001"},
  "payload": {
    "price": {"old": 1.29, "new": 1.49},
    "status": {"old": "active", "new": "updated"}
  }
}
```

3.2.3. Comparison scope

When determining whether a record has changed, the following columns are excluded from comparison:

- Primary key columns — they identify the record, not its content
- Audit columns — `created_at` and `last_updated_at` are set by the database on every upsert and do not reflect a meaningful data change

The source-provided `modification_date` column *is* included in comparisons, as it reflects when the record was last modified in the Vusion system.

3.3. Indexes

Two partial indexes optimise the outbox's two access patterns:

```
CREATE INDEX idx_event_outbox_pending
ON esl.event_outbox (occurred_at) WHERE status = 'PENDING';

CREATE INDEX idx_event_outbox_delivered
ON esl.event_outbox (delivered_at) WHERE status = 'DELIVERED';
```

Index	Purpose
idx_event_outbox_pending	Event Publisher queries: <code>SELECT ... WHERE status = 'PENDING' ORDER BY occurred_at</code>
idx_event_outbox_delivered	Retention cleanup: <code>DELETE ... WHERE status = 'DELIVERED' AND delivered_at < threshold</code>

Partial indexes keep each index small — the pending index only covers rows awaiting delivery, and the delivered index only covers published rows. As events transition from PENDING to DELIVERED, they move from one index to the other.

3.4. Retention

A stored function handles cleanup of delivered events:

```
CREATE OR REPLACE FUNCTION esl.cleanup_delivered_events(retention_days INT DEFAULT 7)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE deleted BIGINT;
BEGIN
    DELETE FROM esl.event_outbox
    WHERE status = 'DELIVERED'
        AND delivered_at < NOW() - (retention_days || ' days')::INTERVAL;
    GET DIAGNOSTICS deleted = ROW_COUNT;
    RETURN deleted;
END; $$;
```

The function deletes outbox rows with status DELIVERED that were published more than retention_days ago (default: 7 days) and returns the number of deleted rows. It is designed to be called externally — by pg_cron, the Event Publisher, or an application-level scheduler.

Only delivered events are eligible for cleanup. Events with status PENDING or FAILED are never deleted, regardless of age, to prevent data loss.

3.5. Migrations

Version	Description
V1.0.0.15	Create event_outbox table
V1.0.0.16	Create partial indexes on event_outbox
V1.0.0.17	Create cleanup_delivered_events retention function

These migrations follow the same conventions established in Phase I: table and indexes in separate files, snake_case naming, all objects in the esl schema.

3.6. Key Design Decisions

3.6.1. Single outbox table for all entity types

All entity types share one `event_outbox` table rather than having per-entity outbox tables. The `entity_type` column distinguishes between them. This simplifies the Event Publisher (one polling query), keeps migration count low, and avoids schema proliferation as new entity types are added.

3.6.2. JSONB for `entity_key` and `payload`

Both `entity_key` and `payload` use JSONB rather than typed columns. This accommodates the varying composite key structures across entity types (2-column key for stores, 3-column key for products) and the variable shape of diff payloads without requiring schema changes.

3.6.3. UUID primary key over sequential integer

A UUID primary key (`gen_random_uuid()`) was chosen over BIGSERIAL to provide deterministic event identifiers. The UUID is included in the published Solace message, enabling downstream consumers to deduplicate events under at-least-once delivery semantics — if the Event Publisher crashes after publishing but before marking the row as DELIVERED, the same event may be published again on restart, and consumers can detect the duplicate by its UUID.

3.6.4. Explicit status column over NULL-based tracking

An explicit status column (PENDING → DELIVERED / FAILED) was chosen over using `delivered_at IS NULL` as the delivery indicator. The explicit model provides clearer semantics, enables a FAILED state for permanent errors, and simplifies observability — `SELECT status, COUNT(*) FROM event_outbox GROUP BY status` gives a complete health snapshot.

3.6.5. Partial indexes over full indexes

Full indexes on `occurred_at` or `delivered_at` would include all rows. Partial indexes are more efficient because the outbox has a natural lifecycle: rows start as PENDING (matched by the pending index) and transition to DELIVERED (matched by the delivered index), then are eventually deleted by the retention function.

3.6.6. Retention via function, not trigger

Unlike Phase I's `sync_state` tables which use AFTER INSERT triggers for retention, the outbox uses an explicitly called function. This avoids adding cleanup overhead to every pipeline transaction — retention runs on its own schedule, independent of the write path.

4. Data Pipeline — CDC

4.1. Overview

Phase 2 adds Change Data Capture (CDC) to the Data Pipeline's PostgreSQL sink. When enabled, the sink detects `CREATED` and `UPDATED` records during each batch write and inserts corresponding change events into the `event_outbox` table — atomically, within the same database transaction as the upserts.

The CDC logic is encapsulated in a dedicated CDC struct that the sink delegates to. When CDC is disabled (the default), the sink behaves exactly as in Phase 1 — no transaction wrapper, no pre-fetch, no outbox writes.

4.2. Feature Flag

CDC is gated behind a single configuration flag:

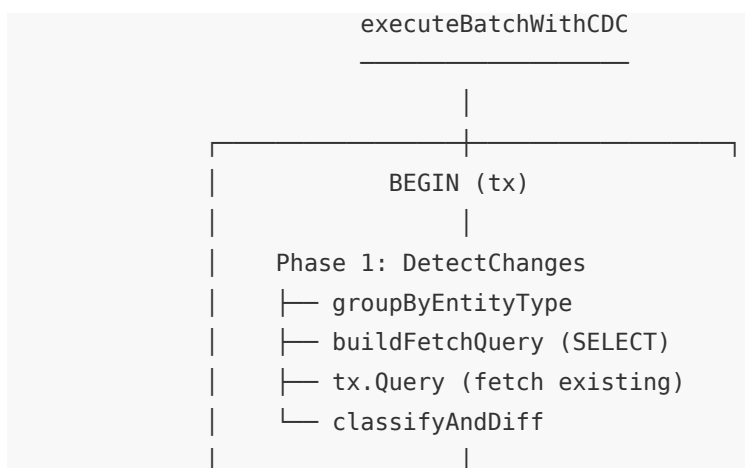
```
sink:
  postgres:
    cdc:
      enabled: false # default
```

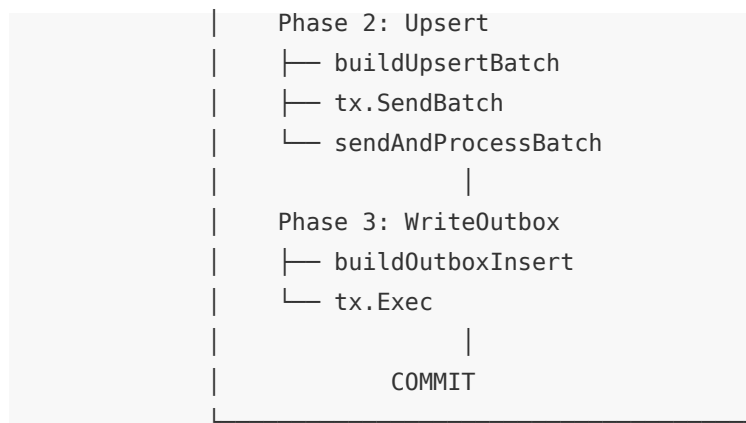
When `cdc.enabled` is `false`, the sink dispatches batches through the original non-transactional path. When `true`, the sink wraps each batch in a transaction and delegates change detection and outbox writes to the CDC module.

The flag is evaluated once at startup in the sink builder — the CDC struct is only instantiated when enabled. At runtime, the dispatch check is a `nil` pointer comparison (`s.cdc != nil`), not a config lookup.

4.3. Architecture

All CDC logic lives in a single file (`internal/sink/postgres/cdc.go`) as an encapsulated struct with two public methods. The sink orchestrates the transaction and delegates each phase:





4.3.1. CDC Struct

```

type CDC struct {
    logger *logger.Logger
    metrics *telemetry.Metrics
}

```

The struct holds no schema reference — the outbox table name is unqualified (the search path handles schema resolution). It exposes two methods:

- **DetectChanges** — fetches existing state within the transaction, classifies each record, and returns a slice of `event.ChangeEvent`
- **WriteOutbox** — inserts change events into the outbox within the same transaction

4.3.2. Shared Helpers

Two functions were extracted from the original `executeBatch` to avoid duplication between the CDC and non-CDC paths:

Helper	Purpose
<code>buildUpsertBatch</code>	Builds a <code>pgx.Batch</code> with upsert queries for all records
<code>sendAndProcessBatch</code>	Processes batch results from any sender (pool or transaction), with a <code>failFast</code> parameter controlling error strategy

The non-CDC path calls these with `failFast=false` (collect all failures). The CDC path calls them with `failFast=true` (return on first error, enabling clean rollback).

4.4. Change Detection

4.4.1. Step 1 — Group by Entity Type

Records in a batch may belong to different entity types (stores, products, labels, access points). The first step groups them using the type field from record metadata:

```
type entityGroup struct {
    tableName    string
    conflictKeys []string
    records      []*models.Record
}
```

Conflict keys come from `entity.ConflictKeys` in `esl-common` — the single source of truth for entity business keys. Records whose entity type is not present in `entity.ConflictKeys` are silently skipped (no business key definition means CDC cannot build a meaningful event).

4.4.2. Step 2 — Fetch Existing State

For each entity group, CDC builds a `SELECT` query that fetches the current row values for all records in the group:

```
SELECT "name"::TEXT, "price"::TEXT, "status"::TEXT, ...
FROM   esl.products
WHERE  ("retail_chain_id", "store_id", "item_id") IN (($1,$2,$3), ($4,$5,$6), ...)
```

Key aspects:

- Explicit column list — derived from the incoming record's `RawData` keys. No `SELECT *` — only columns being upserted are fetched, avoiding false diffs from database-only columns.
- `::TEXT` casting — all values are cast to `TEXT` to avoid Go type mismatches (e.g., `int32` vs `float64`, `time.Time` vs `string`). Both sides are normalised to strings for comparison.
- Single round-trip — one batched `SELECT` per entity type, using a composite `IN` clause.

The query runs within the transaction. Under `READ COMMITTED` isolation (the PostgreSQL default), this is safe because the Data Pipeline is the sole writer to entity tables — no concurrent modifications can occur between the `SELECT` and the subsequent upsert.

4.4.3. Step 3 — Classify and Diff

Each incoming record is compared against the pre-fetched existing state:

Condition	Event	Payload
Key not found in existing rows	CREATED	All non-key fields (full snapshot)
Key found, business fields differ	UPDATED	Only changed fields as {"old": X, "new": Y} diffs
Key found, all fields identical	No event	—

4.4.3.1. Column handling

Column category	Compared?	CREATED payload	UPDATED payload
Conflict keys (e.g., store_id)	No	No (in entity_key)	No (in entity_key)
Audit columns (created_at, last_updated_at)	No	Yes (flat value)	Yes (flat value, not diffed)
Business fields	Yes	Yes	Only changed fields

Audit columns are excluded from comparison because they change on every upsert. However, they are included in the event payload: CREATED events carry both timestamps, and UPDATED events include created_at (from the existing row) and last_updated_at (from the incoming batch) as flat values — not as old/new diffs.

4.4.3.2. Value normalisation

Both sides (incoming RawData values and database TEXT values) are normalised to strings before comparison:

Go type	Normalised form
string	As-is
float64	Formatted as number; trailing .0 stripped for whole numbers
bool	"true" / "false"
nil	"<nil>"
[]any	Sorted, JSON-marshalled
Timestamps	Parsed and normalised (RFC3339, RFC3339Nano, and Postgres TEXT formats are all equivalent)
Postgres arrays	Parsed, sorted, and normalised to a canonical form

This normalisation ensures that semantically equal values are never flagged as changes — for example, a Postgres TEXT timestamp 2024-10-04 08:31:38.879+00 matches an incoming RFC3339 value 2024-10-04T08:31:38.879Z.

4.4.3.3. Timestamp handling in payloads

Values included in event payloads go through a separate normalisation step (`normalizePayloadValue`) that converts Postgres TEXT timestamps to `time.Time` values. When JSON-marshalled for the outbox, these become RFC3339 format — ensuring a consistent representation in published events regardless of how timestamps were originally stored in the database.

4.5. Outbox Write

If any changes were detected, CDC builds a single multi-row INSERT:

```
INSERT INTO event_outbox (event_type, entity_type, entity_key, payload, occurred_at)
VALUES ($1,$2,$3,$4,$5), ($6,$7,$8,$9,$10), ...
```

The `entity_key` and `payload` fields are JSON-marshalled before insertion. The `occurred_at` timestamp is the batch timestamp — a deterministic value set once per batch and injected into all records, ensuring the outbox event and the upserted row share the same audit timestamp.

The `id` (UUID), `status` (PENDING), and `delivered_at` (NULL) columns use their database defaults.

4.6. Transaction Flow

The complete CDC batch execution:

```
BEGIN (READ COMMITTED)
|
|— Phase 1: DetectChanges
|   |— SELECT existing rows (1 query per entity type)
|   |— Classify: CREATED / UPDATED / unchanged
|
|— Phase 2: Upsert
|   |— Build pgx.Batch (1 upsert per record)
|   |— tx.SendBatch → process results (fail-fast)
|   |— On error: ROLLBACK, return failure
|
|— Phase 3: WriteOutbox (only if changes detected)
|   |— INSERT INTO event_outbox (multi-row)
|   |— On error: ROLLBACK, return failure
|
|— COMMIT
```

The transaction guarantees atomicity — if any phase fails, all upserts and outbox writes are rolled back. A record is never persisted without its corresponding change event, and a change event is never written without its corresponding data change.

4.6.1. Single-writer assumption

The pipeline uses READ COMMITTED isolation (PostgreSQL default). This is safe because the Data Pipeline is the sole writer to entity tables. No concurrent modifications can occur between the SELECT (Phase 1) and the upsert (Phase 2), so the pre-fetched state is always current.

If another service starts writing to entity tables in the future, this assumption must be revisited — SERIALIZABLE isolation or explicit row-level locking would be needed to prevent missed diffs.

4.7. Performance

4.7.1. Network round-trips

Path	Round-trips	Description
Non-CDC	1	<code>pool.SendBatch (auto-commit)</code>
CDC	4-5	<code>BEGIN → SELECT → upsert batch → outbox INSERT → COMMIT</code>

The CDC path adds 3-4 round-trips. On a LAN connection, this translates to approximately 0.1ms per additional round-trip. An earlier design combined SELECT and upserts into a single `SendBatch` call to save one round-trip, but this was dropped — the coupling violated single responsibility and saved only ~0.1ms, not worth the complexity.

4.7.2. Measured overhead

Benchmarks against a real PostgreSQL container (batch sizes 10-500):

Scenario	Overhead vs non-CDC
All unchanged (best case)	~30% — SELECT only, no outbox write
Mixed (10% changed)	~50%
All new (worst case)	~100% — every record produces a CREATED event

At `batch_size=500` (the production default), the worst case adds approximately 4 seconds to a full 240k-record sync. This is acceptable for the current workload profile.

If CDC overhead becomes a bottleneck, the following optimisations are available: - Lower batch size to 200-300 (reduces per-batch transaction scope) - Chunk outbox INSERTs for very large batches - Switch to COPY for bulk outbox loading

4.8. Error Handling

4.8.1. Fail-fast semantics

Unlike the non-CDC path (which collects all failures and continues), the CDC path fails fast on the first error. This is required because partial upserts combined with partial outbox writes would be invalid — the transaction must be all-or-nothing.

When a batch fails:

1. The transaction is rolled back (no upserts, no outbox writes)
2. A single `recordFailure` is returned containing the first failing record
3. The error is propagated via `Ack(err)` to all records in the batch
4. The retry mechanism re-attempts the entire batch (up to `max_retries`)

4.8.2. Error classification

The same error classification from Phase I applies:

Type	Action
Transient (deadlock, serialisation, system errors)	Retried with exponential backoff
Permanent (constraint violations, syntax errors)	Stored in <code>records_with_errors</code> , not retried

4.9. Observability

4.9.1. Metrics

Metric	Type	Description
<code>sink.cdc.events</code> (attribute: <code>event_type=CREATED</code>)	Counter	Number of CREATED events detected
<code>sink.cdc.events</code> (attribute: <code>event_type=UPDATED</code>)	Counter	Number of UPDATED events detected
<code>sink.cdc.detect_duration_ms</code>	Histogram	Time spent in change detection (SELECT + classify)
<code>sink.cdc.outbox_duration_ms</code>	Histogram	Time spent writing to the outbox

All metrics include the `sink.type=postgres` attribute and are prefixed with `eslorchestrator.` in the OTel exporter.

4.9.2. Logging

Change detection (INFO level):


```
{
  "msg": "CDC change detection completed",
  "batch_size": 100,
  "cdc_created": 3,
  "cdc_updated": 7,
  "cdc_unchanged": 90,
  "cdc_fetch_ms": 12
}
```

Outbox write (DEBUG level):

```
{
  "msg": "CDC outbox write",
  "event_count": 10,
  "cdc_outbox_ms": 2
}
```

4.10. Configuration Reference

CDC adds a single configuration block under `sink.postgres`:

```
sink:
  postgres:
    # ... existing connection and batching settings ...

    cdc:
      enabled: true # Enable CDC change detection and outbox writes
```

No additional CDC-specific settings are needed. The outbox table must exist in the database (see [Database migrations V1.0.0.15–17](#)).

4.11. Known Limitations

4.11.1. No DELETED event detection

Phase 2 only detects CREATED and UPDATED changes. DELETED event detection is deferred to a future iteration. The `ChangeTypeDeleted` constant is defined in `esl-common` but not used by the CDC logic.

5. Event Publisher

5.1. Overview

The Event Publisher is a long-running Go service that polls the `event_outbox` table for pending CDC events and publishes them to a Solace message broker. It runs as a Kubernetes Deployment, decoupled from the Data Pipeline CronJob.

The service follows a simple loop: `poll` → `transform` → `publish` → `mark delivered`. Each cycle runs within a single database transaction, using `SELECT FOR UPDATE SKIP LOCKED` for row-level locking. Events are published with persistent confirmed delivery, providing at-least-once guarantees.

Event Publisher — Poll Cycle

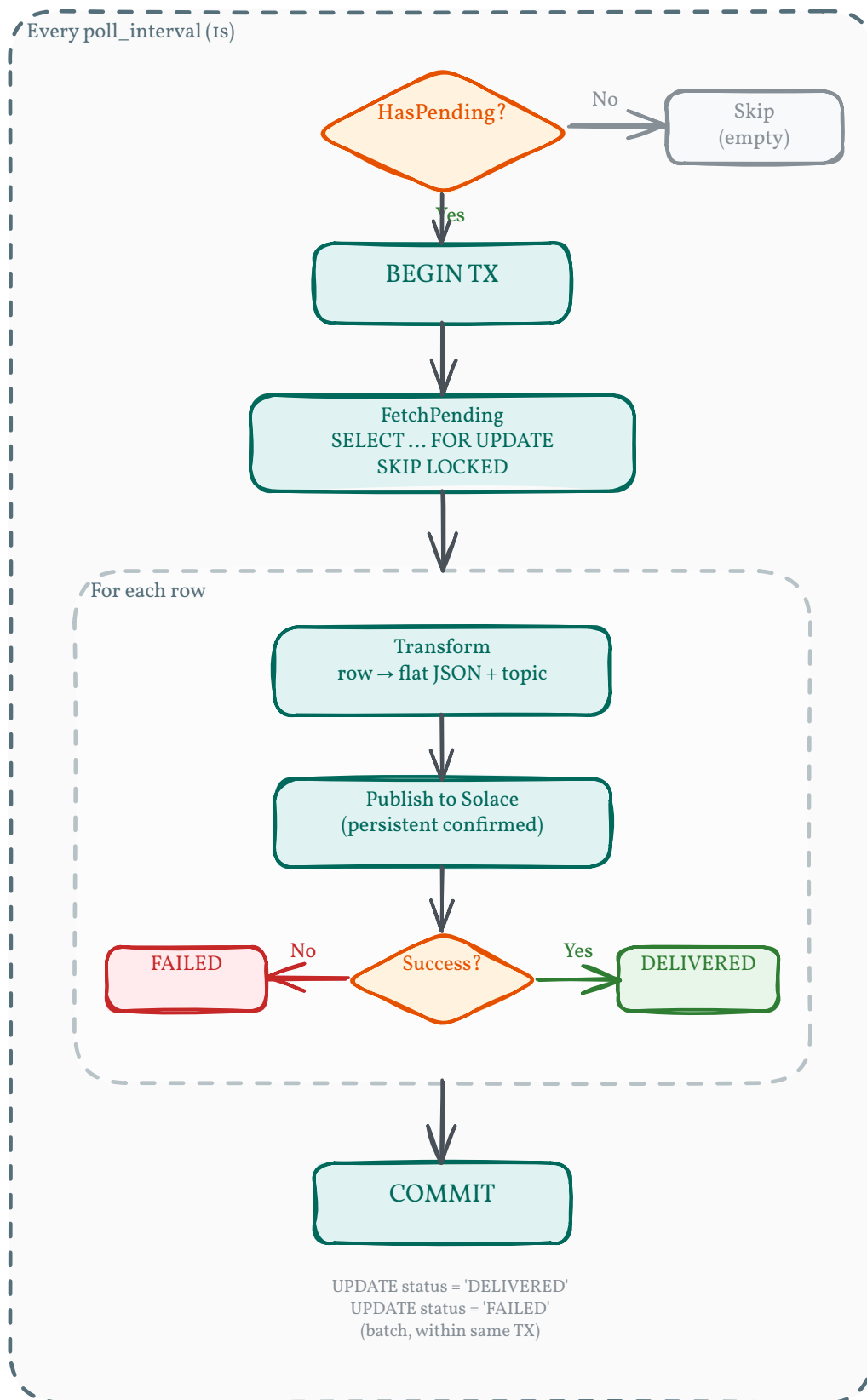


Figure 3: Event Publisher Flow

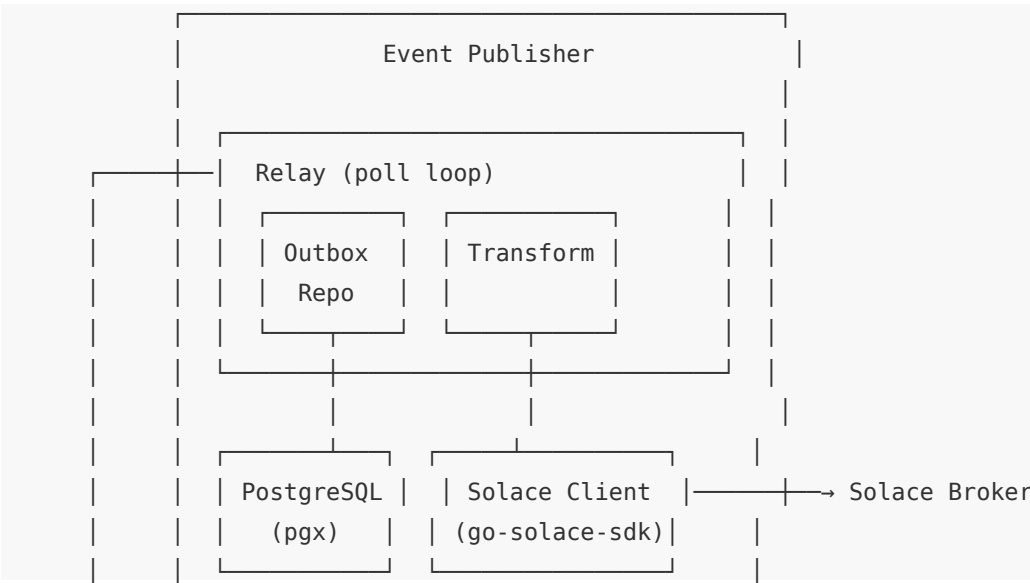
5.2. Technology Stack

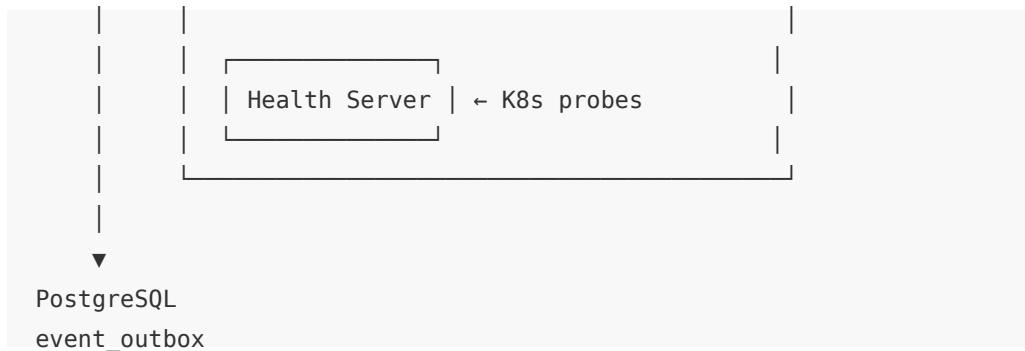
Technology	Purpose
Go 1.26	Application language
pgx/v5 via esl-common	PostgreSQL driver with connection pooling
go-solace-sdk	Solace client library (wraps <code>solace.dev/go/messaging</code>)
Viper	YAML configuration with environment variable overrides
Zap	Structured JSON logging
OpenTelemetry	Metrics (application + SDK-level)

5.3. Architecture

The service is organised into five internal packages:

Package	Responsibility
relay	Core polling loop, batch processing, transaction management
outbox	PostgreSQL repository — fetch pending, mark delivered/failed
transform	Outbox row → flat JSON payload + dynamic Solace topic
messaging/solace	Solace client wrapper — connect, publish confirmed, disconnect
health	HTTP liveness and readiness probe server





5.4. Polling Loop

The relay runs a ticker-based loop at a configurable interval (default: 1 second). Each cycle:

5.4.1. 1. Check for pending events

A cheap `SELECT EXISTS` query determines whether any `PENDING` rows exist. If the outbox is empty, the cycle ends without opening a transaction.

5.4.2. 2. Fetch and lock rows

```

SELECT id, event_type, entity_type, entity_key, payload, status, occurred_at, delivered_at
FROM event_outbox
WHERE status = 'PENDING'
ORDER BY occurred_at
LIMIT $1
FOR UPDATE SKIP LOCKED
  
```

`FOR UPDATE SKIP LOCKED` acquires row-level locks without blocking. Rows already locked by another transaction are skipped, which naturally supports horizontal scaling — multiple publisher replicas can process distinct rows concurrently without coordination.

5.4.3. 3. Process each row

For each fetched row:

1. Transform — Convert the outbox row into a flat JSON payload and a dynamic Solace topic (see [Event Transformation](#))
2. Publish — Send to Solace with persistent confirmed delivery
3. Track outcome — Collect the row ID into a delivered or failed list

Processing continues even if individual rows fail. Transform errors (unsupported entity type, malformed store ID) and publish errors (Solace unreachable, confirmation timeout) both result in the row being marked as `FAILED`, but do not affect other rows in the batch.

5.4.4. 4. Mark outcomes

Two batch updates within the same transaction:

```
UPDATE event_outbox SET status = 'DELIVERED', delivered_at = $1 WHERE id = ANY($2)
UPDATE event_outbox SET status = 'FAILED' WHERE id = ANY($3)
```

5.4.5. 5. Commit

The entire cycle — fetch, status updates — commits atomically. If the transaction fails, all status changes are rolled back and rows return to PENDING for the next cycle.

5.5. Event Transformation

The transform step converts an outbox ChangeEvent into a flat JSON payload and a Solace topic. The Event Publisher does not send raw outbox rows — it restructures the data for downstream consumers.

5.5.1. Payload assembly

1. Start with entity key fields — retail_chain_id, store_id (split — see below), and any additional conflict keys (item_id, label_id, id)
2. Merge payload fields — For CREATED events, the payload is already flat. For UPDATED events, diff objects {"old": X, "new": Y} are flattened to extract only the "new" value. Audit columns (created_at, last_updated_at) pass through as-is (they are already flat values, not diffs).
3. Add metadata — eventId (the outbox row UUID) and send_date (current time in RFC3339)

The result is a flat JSON object regardless of event type. Downstream consumers distinguish between creation and update events via the Solace topic, not the payload structure.

5.5.2. Store ID splitting

The outbox stores composite store IDs in the format {retail_chain_id}.{store_id} (e.g., continente_pt.000010). The transform step splits on the dot and extracts only the store portion (000010) for both the published payload and the topic.

5.5.3. Example published event

```
{
  "eventId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "retail_chain_id": "continente_pt",
  "store_id": "000010",
  "store_name": "Loja Alpha",
  "creation_date": "2026-03-01T10:00:00Z",
  "modification_date": "2026-04-06T14:30:00Z",
  "created_at": "2026-03-01T10:00:00Z",
  "last_updated_at": "2026-04-06T14:30:00Z",
  "send_date": "2026-04-09T12:00:00Z"
}
```

5.6. Solace Publishing

5.6.1. Topic structure

Topics follow a hierarchical pattern:

```
in-store/orchestratoresl/{entitySegment}/{messageType}/v1/{insignia}/{storeId}
```

Variable	Source
{entitySegment}	Static mapping from entity type (see table below)
{messageType}	Event type lowercased: created, updated
{insignia}	retail_chain_id from entity_key
{storeId}	store_id from entity_key (split, store portion only)

Entity type to topic segment mapping:

Entity type	Topic segment
store	stores/store
product	products/product
label	labels/label
accesspoint	accesspoints/accesspoint

Example topics:

```
in-store/orchestratoresl/stores/store/created/v1/continente_pt/000010
in-store/orchestratoresl/products/product/updated/v1/continente_pt/000010
in-store/orchestratoresl/labels/label/created/v1/continente_pt/000010
in-store/orchestratoresl/accesspoints/accesspoint/updated/v1/continente_pt/000010
```

5.6.2. Delivery mode

Events are published using persistent confirmed delivery (`PublishMessageConfirmed`). The Solace broker acknowledges receipt before the publisher considers the message delivered. This prevents data loss during broker-to-consumer distribution.

The `go-solace-sdk` handles publish retries internally (3 attempts, exponential backoff). If all attempts fail or the confirmation timeout is exceeded, the Event Publisher marks the outbox row as `FAILED`.

5.7. Health Endpoints

The service exposes HTTP endpoints for Kubernetes probes:

Endpoint	Probe	Behaviour
/health	Liveness	Always returns 200 OK if the process is running
/ready	Readiness	Returns 200 OK if both PostgreSQL and Solace are connected; 503 Service Unavailable otherwise

The readiness check pings the database pool and verifies the Solace client's connection status. If either dependency is down, the service reports not ready and Kubernetes stops routing traffic to it.

Port and paths are configurable via `health.port`, `health.health_path`, and `health.ready_path`.

5.8. Graceful Shutdown

On SIGINT or SIGTERM:

1. Stop accepting new poll cycles
2. Finish the current batch (publish remaining events, update statuses, commit)
3. Shut down the health HTTP server
4. Disconnect the Solace client
5. Close the database pool

A configurable shutdown timeout (`publisher.shutdown_timeout`, default: 30 seconds) bounds the entire sequence. If exceeded, the process force-exits.

5.9. Configuration Reference

The service is configured via a YAML file with environment variable overrides. Environment variables use uppercase underscore notation (e.g., `DATABASE_HOST`, `SOLACE_TOPIC_PREFIX`).

5.9.1. Database

database:

```
host: localhost
port: 5432
user: esl
password: ""
database: eslorchestrator
schema: esl                                # Sets search_path at pool level
ssl_mode: disable
application_name: "event-publisher"
max_conns: 4
min_conns: 1
max_conn_lifetime: "1h"
max_conn_idle_time: "30m"
```



```
health_check_period: "1m"
connect_timeout: "5s"
```

All fields map directly to `esl-common's postgres.PoolConfig`.

5.9.2. Solace

solace:

```
host: tcp://localhost:55555      # tcp:// or tcps:// for TLS
vpn: default                    # Message VPN name
username: admin
password: admin
topic_prefix: "in-store/orchestratoresl"
connect_timeout: "10s"          # TCP connection timeout (min: 1s)
connect_retries: 3              # TCP connect retry attempts
reconnect_retries: 3            # Post-disconnect reconnection attempts (-1 = infinite)
reconnect_wait: "2s"           # Delay between reconnect attempts (min: 100ms)
keep_alive: "30s"              # TCP keep-alive interval
confirmation_timeout: "10s"     # Publish confirmation timeout
```

5.9.3. Publisher

publisher:

```
poll_interval: "1s"            # Outbox polling frequency
batch_size: 100                # Max events per poll cycle
shutdown_timeout: "30s"        # Max time to finish batch on shutdown
```

5.9.4. Health

health:

```
port: 8081
health_path: "/health"
ready_path: "/ready"
```

5.9.5. Logging

log:

```
level: info                    # debug, info, warn, error
```

5.10. Observability

5.10.1. Application metrics

Metric	Type	Attributes	Description
event_publisher.events_processed_total	Counter	entity_type, event_type, status	Events processed, segmented by outcome
event_publisher.poll_batch_size	Histogram	—	Events fetched per poll cycle
event_publisher.poll_cycles_total	Counter	—	Total poll cycles (including empty)

The status attribute on `events_processed_total` distinguishes between delivered and failed outcomes, enabling alerting on failure rates per entity type.

5.10.2. SDK metrics

The `go-solace-sdk` automatically records its own metrics when a `MeterProvider` is supplied:

- `solace.core.*` — Connection pool and transport metrics
- `solace.producer.*` — Publish confirmation counts, latency, retries, and payload size

5.10.3. Logging

All log entries use Zap structured JSON. Key log events:

Level	Event	Fields
INFO	Poll cycle completed	batch_size, delivered, failed
INFO	Startup / shutdown	Component states
WARN	Publish failure	event_id, entity_type, error
WARN	Transform failure	event_id, entity_type, error
ERROR	Transaction failure	error
DEBUG	Individual event processed	event_id, topic, status

The `go-solace-sdk` logs are bridged through Zap via a `zapslog.NewHandler` adapter, consolidating all output into a single structured format.

5.11. Delivery Guarantees

The Event Publisher provides at-least-once delivery:

1. Events are published to Solace before the database status is updated
2. If the process crashes between publishing and committing, the outbox row stays PENDING
3. On restart, the row is polled again and re-published

Consumers must deduplicate using the `eventId` field in each published event. This is the outbox row's UUID primary key — deterministic across retries. The duplication window is bounded by the batch size (default: 100 events) and only occurs during crash recovery, not during normal operation.

5.12. Key Design Decisions

5.12.1. Polling over WAL logical replication

The Event Publisher polls the outbox table rather than consuming PostgreSQL's WAL (Write-Ahead Log) via logical replication. Both approaches were evaluated:

Concern	Polling	WAL streaming
Operational complexity	Low — standard SQL queries	High — requires <code>wal_level=logical</code> , replication slots, WAL retention management
Latency	Configurable (default is)	Near real-time (sub-second)
Observability	Free via status column — GROUP BY status shows outbox health	Requires external monitoring of replication lag and slot state
WAL bloat risk	None	Inactive slot holds WAL indefinitely, risking disk exhaustion
Failure handling	Explicit FAILED status in database	Complex — failed messages need a dead-letter mechanism
Scaling	FOR UPDATE SKIP LOCKED supports multiple replicas	Requires partitioned replication or a single consumer

Polling was chosen because the 1-second latency is sufficient for the ESL use case, operational simplicity is a priority (no DBA involvement for replication slots), and the explicit status column provides built-in observability. WAL streaming would be the better choice for high-throughput or sub-second latency requirements.

5.12.2. Transaction-scoped publishing

The entire poll cycle — fetch, publish, mark — runs within a single database transaction. Rows stay locked during Solace publishing. This is acceptable because:

- The Data Pipeline only INSERTs into the outbox (no conflict with row locks)
- Batch size is capped (100 events)
- A single publisher instance is expected in the initial deployment

5.12.3. No application-level retry for failed events

Events that fail to transform or publish are marked FAILED immediately — there is no automatic retry. The `go-solace-sdk` handles low-level publish retries internally (3 attempts, exponential backoff), but if the SDK exhausts its retries, the event is considered failed.

This design keeps the publisher stateless and simple. Failed events remain in the outbox for investigation. Replay or retry is handled externally (e.g., resetting status to PENDING after fixing the root cause).

5.13. Known Limitations

5.13.1. No per-entity ordering guarantee

With a single publisher replica, events for the same entity are published in `occurred_at` order. With multiple replicas, `FOR UPDATE SKIP LOCKED` distributes rows across replicas without entity-key affinity — two events for the same entity could be processed by different replicas and arrive out of order. If downstream consumers require per-entity ordering, partitioning by entity key (e.g., consistent hashing) would be needed.

5.13.2. Failed events require manual intervention

Events marked `FAILED` stay in the outbox indefinitely. There is no dead-letter queue or automatic retry mechanism. Recovery requires identifying the root cause and resetting the status to `PENDING` manually or via an operational script.